HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN I

BÁO CÁO TUẦN 2 THỰC TẬP CƠ SỞ

Đề tài: HỆ THỐNG QUẢN LÝ TÀI CHÍNH CÁ NHÂN THÔNG MINH TÍCH HỢP PHÂN TÍCH HÀNH VI VÀ DỰ BÁO CHI TIÊU

Giảng viên hướng dẫn : **Kim Ngọc Bách**
Họ và tên sinh viên : **Nguyễn Thị Diệu Ánh**
Mã sinh viên : **B23DCCN062**

Hà Nội – 2026

1. mục tiêu

- thiết lập môi trường phát triển backend
- cài đặt framework FastAPI
- thiết kế API cho hệ thống
- Xây dựng các chức năng CRUD cơ bản.

2. Các công nghệ sử dụng

- Python: ngôn ngữ.
- FastAPI: Framework xây dựng API
- Uvicorn: ASGI server chạy API
- SQLAlchemy: ORM kết nối database.
- Cơ sở dữ liệu
- Pydantic: Validation dữ liệu

3. Cài đặt môi trường phát triển

- Cài đặt các thư viện cần thiết

```
smart-finance-api/
├── backend/
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   ├── schemas.py
│   ├── crud.py
│   └── routers/
│       ├── transactions.py
│       ├── budgets.py
│       ├── dashboard.py
│       ├── report.py
│       └── forecast.py
└── requirements.txt
```

4. Mô tả chi tiết mã nguồn hệ thống

4.1. Cấu hình kết nối Cơ sở dữ liệu (database.py)

Sử dụng thư viện **SQLAlchemy** để thiết lập kết nối tới cơ sở dữ liệu MySQL. Hệ thống sử dụng cơ chế SessionLocal để quản lý các phiên làm việc và Base class để định nghĩa các mô hình thực thể.

- **Chức năng:** Khởi tạo kết nối, quản lý pool kết nối và cung cấp hàm get_db() để thực hiện Dependency Injection trong các Router.

4.2. Định nghĩa Mô hình dữ liệu (models.py)

Chuyển đổi sơ đồ ERD từ Tuần 1 thành các lớp Python (Classes). Các thuộc tính được định nghĩa rõ ràng về kiểu dữ liệu và mối quan hệ:

- **Mối quan hệ 1-N:** Giữa User với Transactions, Budget, Notifications.
- **Mối quan hệ 1-1:** Giữa Transactions với Emotion_Log và NLP_Logs để phục vụ việc phân tích AI sau này.
- **Ràng buộc:** Sử dụng ForeignKey để đảm bảo tính toàn vẹn dữ liệu giữa các bảng.

4.3. Phân lớp API với APIRouter (routers/)

Để mã nguồn dễ bảo trì và mở rộng, hệ thống được chia nhỏ thành các module định tuyến:

- **dashboard.py**: Chứa các Endpoint cung cấp dữ liệu tổng hợp cho giao diện Overview như số dư (Balance), tổng thu chi trong tháng và dữ liệu biểu đồ cảm xúc.
- **transactions.py**: Quản lý luồng dữ liệu nghiệp vụ chính, cho phép người dùng thêm mới giao dịch từ câu lệnh tự nhiên (Smart Input) và lưu trữ vào Database.
- **budgets.py**: Xử lý việc thiết lập và theo dõi hạn mức chi tiêu.

4.4. Logic nghiệp vụ và Xử lý dữ liệu (crud.py)

Đây là tầng xử lý logic cốt lõi của hệ thống (Create, Read, Update, Delete):

- **Tính toán số dư:** Hàm get_dashboard thực hiện truy vấn tổng hợp dữ liệu từ bảng Transactions theo tháng hiện tại của người dùng.
- **Tự động hóa cảnh báo:** Khi một giao dịch được thêm vào, hệ thống thực hiện kiểm tra ngân sách của danh mục tương ứng. Nếu tổng chi vượt quá hạn mức (limit_amount), một bản ghi cảnh báo sẽ tự động được ghi vào bảng Notifications.
- **Truy vấn theo cảm xúc:** Hàm get_spending_by_emotion thực hiện phép JOIN giữa bảng Transactions và Emotion_Log để thống kê chi phí theo trạng thái tâm lý.

4.5. Kiểm soát dữ liệu đầu vào (schemas.py)

Sử dụng **Pydantic** để định nghĩa các bản mẫu dữ liệu (Schemas):

- Đảm bảo mọi giao dịch gửi lên phải có amount là kiểu số thực dương.
- Định nghĩa cấu trúc dữ liệu trả về (Response Model) để đảm bảo API chỉ cung cấp những thông tin cần thiết, giúp tăng tính bảo mật và tối ưu tốc độ truyền tải.

Dashboard

GET	/dashboard/	Dashboard

Transactions

GET	/transactions/	Read Transactions
POST	/transactions/	Create Transaction
DELETE	/transactions/{transaction_id}	Delete Transaction
POST	/transactions/analyze	Analyze Transaction

Budgets

POST	/budgets/	Create Budget
GET	/budgets/monthly	Get Monthly Budget
GET	/budgets/progress	Category Progress
GET	/budgets/overview	Budget Overview

Reports

GET	/reports/dashboard	Dashboard
GET	/reports/category	Expense By Category
GET	/reports/emotion	Emotion Spending
GET	/reports/monthly	Monthly Report

Forecast

GET	/forecast/	Forecast